

GeM TenderLens — Project Documentation

GeM TenderLens

Multi-Agent GenAI Procurement Evaluation System for Government e-Marketplace Tenders

1. Project Overview

Project Title

GeM TenderLens - Multi-Agent GenAI Procurement Evaluation System

Objective

The objective of this project is to build an AI-powered multi-agent platform that automates the evaluation of Government e-Marketplace (GeM) tender proposals by indexing governing tender documents, ingesting vendor submissions, verifying technical compliance against mandatory clauses, normalizing commercial pricing, identifying procurement risks, and generating audit-ready evaluation reports. The platform enables procurement committees to compare multiple vendor bids against a tender's requirements quickly, with defensible, citation-backed evidence for every finding.

Business Problem

Evaluating procurement proposals for government tenders requires cross-referencing complex Technical Specification Documents, Bills of Quantities, commercial price schedules, GST rules, warranty terms, and legal corrigenda. Vendors submit proposals in heterogeneous formats such as emails, PDF quotations, Word documents, and Excel BOQs, making manual comparison against a lengthy tender document slow and labour-intensive. Vendors also sometimes include non-compliant terms in fine print, such as requesting an advance payment the tender forbids, or offering a shorter warranty period than

what is mandatory, and these are easily missed during manual review. In addition, traditional spreadsheet-based evaluation summaries do not carry exact page or clause citations, making it difficult for an audit team to later verify why a vendor was qualified or disqualified. This project addresses these problems by automating document ingestion, evidence-backed compliance checking, and commercial evaluation using a combination of retrieval-augmented generation and multi-agent reasoning.

Expected Outcomes

- Automated ingestion and clause-level indexing of GeM tender documents.
 - Structured extraction of vendor pricing, delivery, warranty, and certification data.
 - Evidence-backed technical compliance verification against mandatory tender requirements.
 - Commercial price normalization with GST handling and L-1 (lowest bidder) determination.
 - Automated risk and clarification queue for missing or unverifiable vendor claims.
 - Committee-ready evaluation reports exported in Markdown, HTML, JSON, and PDF.
 - Improved evaluation speed, consistency, and audit traceability for procurement committees.
-

2. Data Sources and Knowledge Base Documents

Document Corpus Name

GeM Tender and Vendor Proposal Document Set

Source

The documents used in this project follow the structure of real GeM tender packages and vendor proposal submissions. Governing tender packages and vendor proposals used for development and testing were prepared to closely

resemble authentic GeM procurement documentation, including realistic clause structures, BOQ formats, and commercial quotation language, while avoiding the use of any confidential or proprietary business information.

Document Corpus Overview

The knowledge base consists of two categories of documents that together represent a complete tender evaluation cycle: the governing tender package uploaded by the procurement team, and the vendor proposal submissions received against that tender. Both categories are parsed, chunked at the clause level, converted into vector embeddings, and stored in ChromaDB, enabling semantic retrieval through a Retrieval-Augmented Generation pipeline for both compliance verification and natural-language clause search.

Document Types Used

Document Type	Description
Tender Bid Document (PDF, TXT)	The governing GeM tender or bid document containing eligibility criteria, terms, and clause references.
Technical Specification (PDF, DOCX)	Line-item technical requirements and mandatory product or service specifications.
Bill of Quantities (XLSX, CSV)	Structured bill-of-quantities line items covering quantities, categories, and specifications.
Corrigenda and Clarifications (PDF, TXT)	Amendments, clarifications, and updates issued after the original tender was published.
Vendor Proposal Email (.eml)	Vendor proposal submissions received as email, including quoted pricing and technical claims.
Vendor Technical Brochure (PDF, DOCX)	Supporting vendor documentation describing offered products, certifications, and compliance claims.

Purpose of the Documents

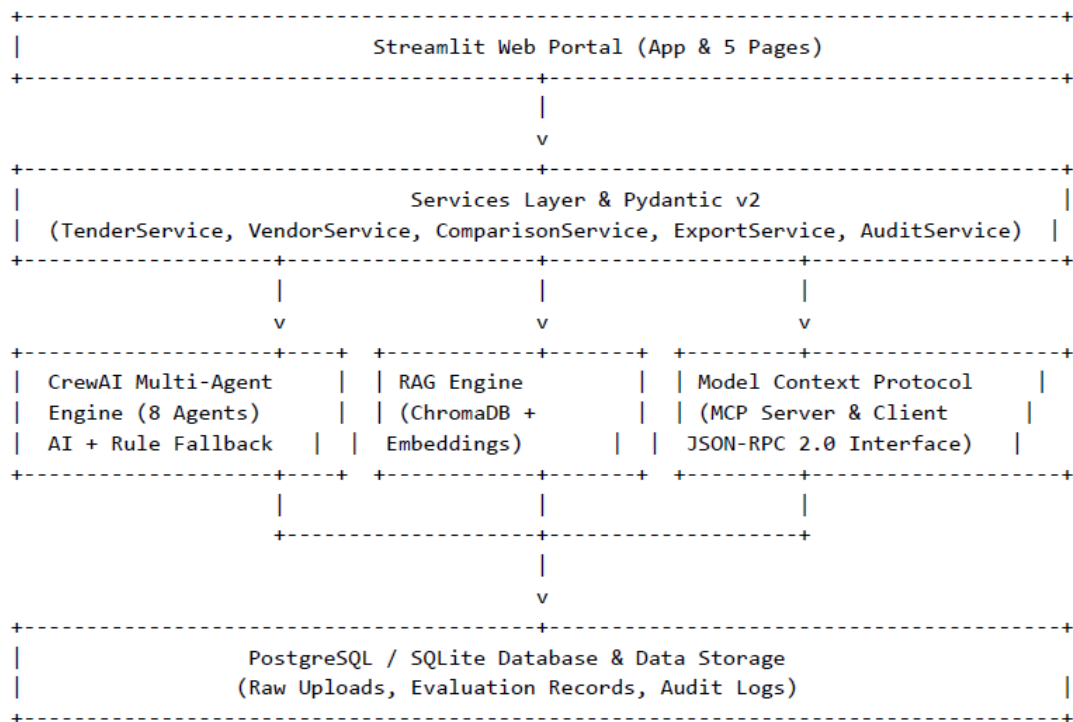
The document corpus enables the platform to:

- Retrieve exact tender clauses and requirement text through the RAG pipeline.
- Compare vendor technical claims against mandatory tender specifications.

- Normalize and compare commercial pricing, GST treatment, and delivery terms across vendors.
- Identify missing documentation, unverifiable claims, and compliance risks.
- Generate evidence citations that reference the exact source file, page, and clause for every finding.

3. System Architecture

GeM TenderLens is built using a multi-agent AI architecture that automates end-to-end tender evaluation. The platform uses a Streamlit web portal for the user interface, a services layer built on Pydantic v2 for structured business logic, CrewAI to orchestrate multiple specialized AI agents, ChromaDB with Cohere and SentenceTransformer embeddings as the retrieval knowledge base, Mistral AI and Groq for language-model reasoning, and PostgreSQL with a SQLite fallback for transactional and audit data. A Model Context Protocol server exposes the platform's core capabilities through a standardized JSON-RPC 2.0 interface.



System Architecture Diagram

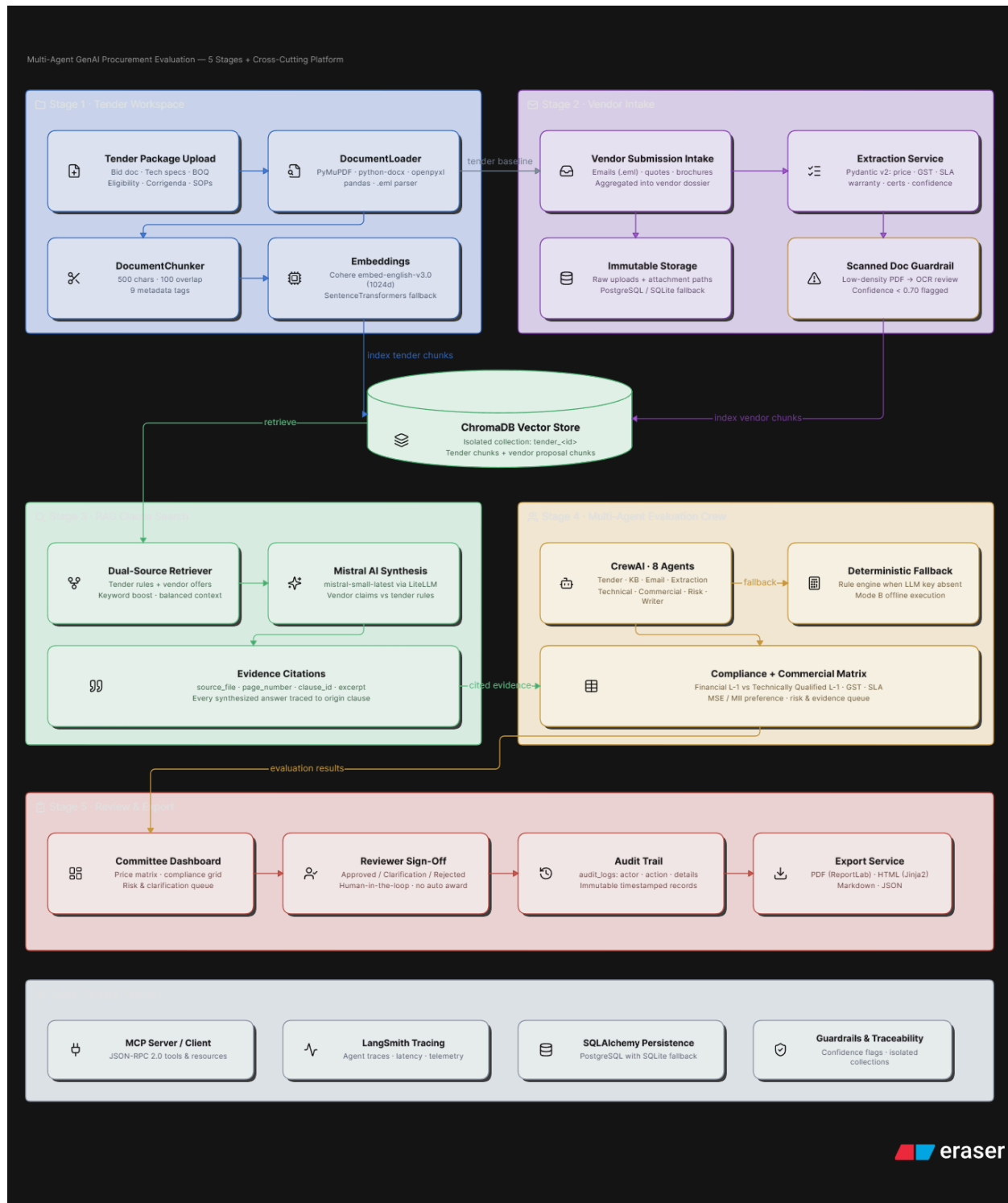
Technology Stack

Component	Technology	Description
Frontend UI	Streamlit 1.30+	Five-screen interactive procurement workspace with session-state preservation.
Language	Python 3.11	Core business logic, document parsing, and orchestration.
Multi-Agent Framework	CrewAI	Specialized agent roles, sequential task assembly, and execution.
Data Validation	Pydantic v2	Type-safe schemas for tenders, submissions, proposals, and findings.
Vector Database	ChromaDB	Isolated per-tender collections for clause and proposal retrieval.
Embeddings	Cohere / Sentence-Transformers	Semantic embeddings with automatic local fallback.
LLM Orchestration	Mistral AI / Groq	Multi-agent reasoning, field extraction, and RAG answer synthesis.
Document Parsing	PyMuPDF, python-docx, openpyxl, pandas	Ingests PDF, DOCX, XLSX, CSV, TXT, and email files.
Transactional Database	PostgreSQL / SQLite	SQLAlchemy ORM with automatic table creation and fallback.
Export Service	ReportLab, Markdown, JSON	Generates committee-ready reports in four formats.
External Interface	Model Context Protocol	JSON-RPC 2.0 standardized tool and resource interface.
Telemetry	LangSmith	Multi-agent trace logging and LLM call observability.

Key Components

- **Streamlit Web Portal** — receives procurement-committee interactions and renders the five-stage workflow.
- **Services Layer** — coordinates tender ingestion, vendor intake, comparison, export, and audit logging.

- **CrewAI Multi-Agent Engine** — performs specialized tasks including selection, compliance, commercial analysis, risk assessment, and reporting.
 - **RAG Engine (ChromaDB and Embeddings)** — stores and retrieves tender and vendor document chunks with evidence citations.
 - **Model Context Protocol Server** — exposes tool and resource access through a standard JSON-RPC 2.0 interface.
 - **PostgreSQL / SQLite Database** — persists dynamically extracted requirements, audit logs, and reviewer decisions.
-



Flow Diagram — Multi-Agent GenAI Procurement Evaluation

4. Project Directory Structure

```
Gem_Tender_Project/
├─ app.py
├─ mcp_server.py
├─ mcp_client.py
├─ utils_logger.py
├─ requirements.txt
├─
├─ crew/
│   ├─ agents.py
│   ├─ tasks.py
│   ├─ tender_crew.py
│   └─ tools.py
├─
├─ rag/
│   ├─ chroma_client.py
│   ├─ chunking.py
│   ├─ document_loader.py
│   ├─ embeddings.py
│   └─ retriever.py
├─
├─ schemas/
│   ├─ audit.py
│   ├─ evaluation.py
│   ├─ tender.py
│   └─ vendor.py
├─
├─ services/
│   ├─ audit_service.py
│   ├─ comparison_service.py
│   ├─ database.py
│   ├─ db_models.py
│   ├─ export_service.py
│   ├─ tender_service.py
│   ├─ validation_service.py
│   └─ vendor_service.py
```



```
|
| └─ utils/
|   │ └─ evaluator_helper.py
|   │ └─ gst_helper.py
|   │ └─ proposal_extractor.py
|   │ └─ session_state.py
|   │ └─ status_badges.py
|
| └─ pages/
|   │ └─ 1_tender_workspace.py
|   │ └─ 2_vendor_intake.py
|   │ └─ 3_rag_search.py
|   │ └─ 4_vendor_comparison.py
|   │ └─ 5_review_export.py
|
| └─ docs/
```

5. Multi-Agent Architecture

GeM TenderLens follows a multi-agent architecture in which each AI agent is responsible for a specific stage of the tender evaluation workflow. This modular design allows each agent to focus on a specialized task, improves maintainability, and allows the platform to fall back to a deterministic rule-based engine for any stage when a language-model connection is unavailable.

Tender Selection Agent

The Tender Selection Agent confirms the active governing tender package and accounts for any corrigenda or amendments that have been issued. It establishes the baseline document set against which all vendor proposals will be evaluated.

Responsibilities:

- Identify the governing GeM tender package.
- Confirm the active tender version.
- Account for corrigenda and amendments.

Knowledge Base Agent

The Knowledge Base Agent is responsible for chunking tender clauses and populating metadata-rich vectors in ChromaDB, ensuring every stored chunk carries the citation information needed for evidence-backed retrieval.

Responsibilities:

- Chunk tender clauses at the appropriate granularity.
- Attach metadata tags including page number and clause identifier.
- Maintain isolated ChromaDB collections per tender.

Email Intake Agent

The Email Intake Agent organizes incoming vendor proposal emails and attachments, establishing a structured submission record for each vendor.

Responsibilities:

- Organize vendor proposal emails and attachments.
- Establish vendor submission dossiers.
- Track submission metadata such as sender and subject.

Extraction Agent

The Extraction Agent pulls structured financial, commercial, and technical fields out of vendor submissions, including quoted price, tax treatment, delivery timelines, and warranty terms.

Responsibilities:

- Extract quoted price and tax figures.
- Extract delivery and warranty terms.
- Extract technical claims and certificate references.

Technical Compliance Agent

The Technical Compliance Agent maps vendor technical evidence against the mandatory tender specifications. It distinguishes between a verifiable compliance

claim, such as one that includes a certificate number or reference ID, and a general claim without supporting evidence.

Responsibilities:

- Map vendor specifications against mandatory requirements.
- Verify whether supporting evidence is present for each claim.
- Flag unverifiable claims for committee review.

Commercial Analysis Agent

The Commercial Analysis Agent normalizes vendor price quotes, tax treatment, delivery lead times, and payment terms into a comparable cost matrix, and determines the L-1, or lowest-cost, ranking among vendors.

Responsibilities:

- Normalize base price and GST treatment across vendors.
- Rank vendors by total normalized cost.
- Determine the lowest financial bid and the lowest technically-qualified bid.

Risk and Evidence Agent

The Risk and Evidence Agent identifies compliance gaps, unverified claims, missing certificates, and low-confidence extractions, and raises formal clarification items for the procurement committee.

Responsibilities:

- Identify compliance gaps and missing documentation.
- Flag low-confidence extractions for manual review.
- Compile a risk and clarification queue.

Evaluation Writer Agent

The Evaluation Writer Agent synthesizes the findings from all preceding agents into a neutral, evidence-backed evaluation report with exact page and clause citations, ready for committee review.

Responsibilities:

- Synthesize findings from all agents into one report.
- Preserve exact evidence citations for every finding.
- Present a neutral, committee-ready summary.

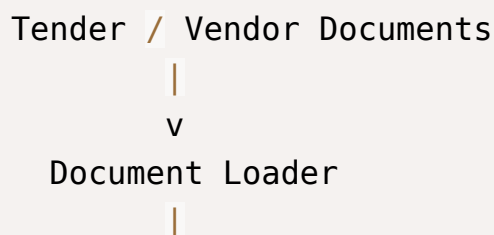
Dual-Mode Execution

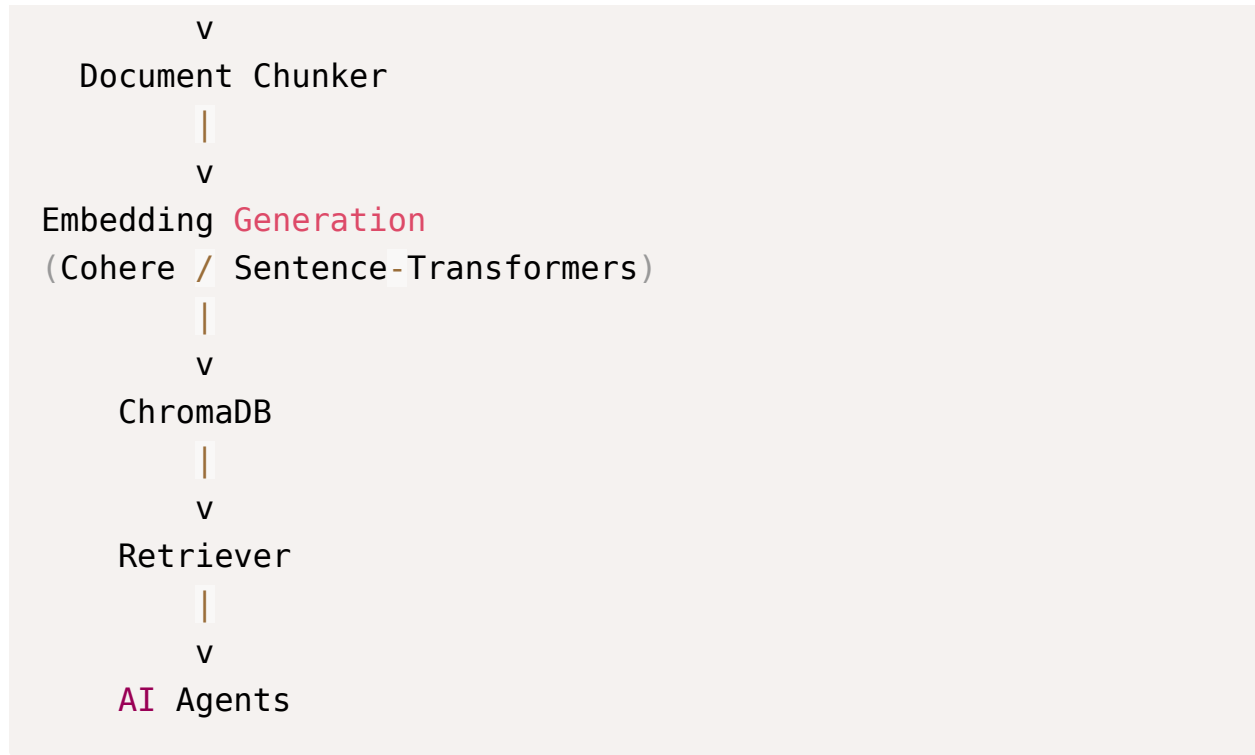
The evaluation workflow supports two execution modes. In Mode A, the CrewAI multi-agent workflow is used whenever a valid language-model API key, such as a Mistral or Groq key, is configured. In Mode B, a deterministic rule-based engine performs the same evaluation using structured comparison rules whenever the platform is offline or no API key is available, ensuring the evaluation pipeline always produces a result.

6. Knowledge Base (Retrieval-Augmented Generation)

The knowledge base provides the contextual information required by the AI agents to perform tender and proposal evaluation. It consists of the governing tender documents and the vendor proposal submissions associated with each tender, isolated into a dedicated vector collection so that data from one tender is never mixed with another.

Documents are parsed according to their format, split into clause-level chunks with page and clause metadata attached, and converted into vector embeddings using the Cohere API, with an automatic local SentenceTransformers fallback when the API is unavailable. These embeddings are stored in ChromaDB. When an agent or a reviewer query requires information, the system retrieves the most relevant chunks through a Retrieval-Augmented Generation pipeline, combining vector similarity with keyword-based re-ranking, so that responses are grounded in the actual tender and vendor documents rather than relying only on the language model's pre-trained knowledge.





7. Core System Modules and Services

Data Schemas

Built with Pydantic v2 to ensure type safety across the platform:

- **tender.py:** TenderRequirement (clause identifier, category, mandatory status) and TenderPackage (metadata, documents, requirements).
- **vendor.py:** VendorSubmission (vendor details, timestamps, attachment paths) and VendorProposal (quoted amount, tax, delivery, warranty, technical claims, certificates).
- **evaluation.py:** EvidenceCitation (file, page, clause identifier, excerpt) and EvaluationFinding (status: compliant, non-compliant, review required).
- **audit.py:** AuditLog (action type, actor, timestamp, details).

Core Services Layer

- **tender_service.py:** Registers tender packages, extracts requirements, handles corrigenda updates, and updates the vector store index.

- **vendor_service.py:** Ingests vendor submissions, runs proposal field extraction, and builds structured vendor dossiers.
- **comparison_service.py:** Normalizes commercial pricing, computes financial and technically-qualified L-1 rankings, evaluates preference rules, and generates the technical compliance matrix and risk queue.
- **export_service.py:** Exports evaluation reports in Markdown, HTML, JSON, and PDF formats.
- **audit_service.py:** Records procurement audit events into PostgreSQL or SQLite.
- **validation_service.py:** Enforces file-accessibility and extraction-confidence checks.
- **database.py and db_models.py:** Manage SQLAlchemy ORM models with automatic table creation and SQLite fallback.

Model Context Protocol

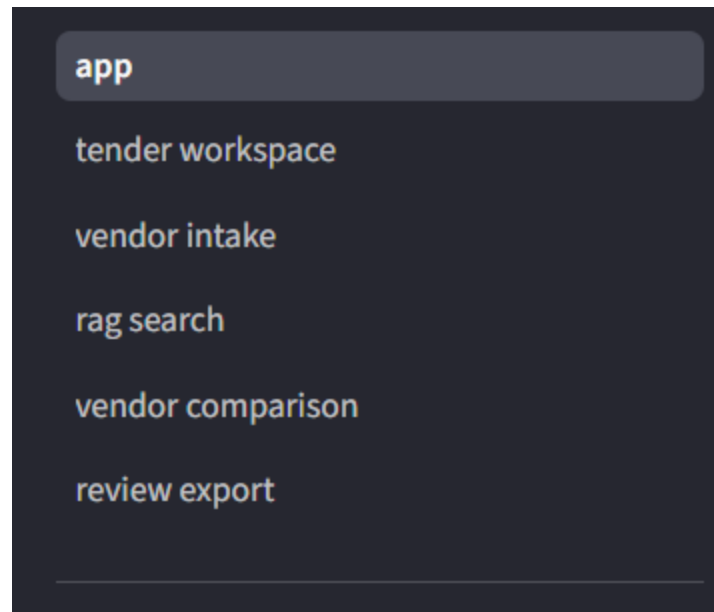
The platform implements the Model Context Protocol through a standard JSON-RPC 2.0 interface, exposing the following operations:

- **tools/list:** Exposes registered tools, including vector search, proposal extraction, vendor email parsing, and tender listing.
- **tools/call:** Executes a requested tool with the supplied input arguments.
- **resources/list and resources/read:** List and retrieve tender package resources.

Utility Helpers

- **proposal_extractor.py:** Standardized extraction of prices, GST, delivery days, warranty terms, and certificates.
 - **evaluator_helper.py:** Rule-based technical evaluation logic.
 - **gst_helper.py:** GST rate normalization and inclusion checks.
 - **session_state.py:** Centralized Streamlit session-state management.
 - **status_badges.py:** Status labels used across the review dashboard.
-

8. User Interface and Five-Stage Procurement Workflow



Five-Stage Procurement Workflow

1. **Page 1: Tender Workspace:** Upload or select GeM tender documents, view the requirement matrix, upload corrigenda, and trigger vector indexing.
2. **Page 2: Vendor Intake:** Parse incoming vendor emails or proposal files, review extracted prices and taxes, and build vendor dossiers.
3. **Page 3: Tender RAG Search:** Perform natural-language queries against indexed tender clauses and vendor claims with full evidence citations.
4. **Page 4: Vendor Comparison:** Trigger multi-agent or rule-based evaluation, compare financial versus qualified L-1 bids, and review risk flags.
5. **Page 5: Review and Export:** Record committee decisions, inspect the timestamped audit trail, and download evaluation packages.

9. Database Schema and Audit Logging

Database tables are defined in `services/db_models.py`:

Table	Key Fields
tenders	tender_id (primary key), title, category, status, created_at, requirements, documents
vendor_submissions	id (primary key), vendor_id, vendor_name, tender_id, received_at, proposal data, attachment paths
audit_logs	log_id (primary key), timestamp, actor, action_type, details

10. Challenges

During the development of GeM TenderLens, several technical and architectural challenges were encountered.

Challenges Faced

- Designing a clause-aware document chunker that keeps related sub-items, such as table rows and specification bullets, grouped under their parent clause instead of fragmenting them.
- Coordinating a multi-agent workflow using CrewAI while ensuring a reliable deterministic fallback when no language-model API key is configured.
- Building a dual-source Retrieval-Augmented Generation pipeline that balances governing tender rules against vendor proposal content during search.
- Normalizing commercial pricing correctly when vendors quote prices as GST-exclusive rather than providing a final inclusive figure.
- Detecting scanned or image-only pages reliably and routing them for manual review instead of silently omitting their content.
- Wiring custom tools correctly into the CrewAI agent framework so that agents can query the knowledge base directly rather than relying only on pre-fetched context.
- Ensuring extracted vendor data, such as warranty duration and delivery timelines, is parsed consistently across differently worded proposal documents.
- Maintaining a single, consistent audit trail across both the AI-driven and rule-based evaluation execution paths.

11. Learnings

This project provided practical experience in developing an enterprise-style AI application using modern retrieval and multi-agent frameworks.

Key Learnings

- Learned to design and implement a multi-agent AI system using CrewAI, including sequential task assembly and a deterministic fallback path.
- Gained hands-on experience building a Retrieval-Augmented Generation pipeline using ChromaDB, Cohere embeddings, and a local embedding fallback.
- Improved understanding of structured data validation and schema design using Pydantic v2 across a multi-service architecture.
- Developed skills in transactional data modeling with SQLAlchemy, including automatic database creation and safe fallback handling.
- Learned to implement the Model Context Protocol for standardized, JSON-RPC 2.0 tool and resource exposure.
- Gained experience in real-world domain modeling for government procurement, including GST normalization and lowest-bidder versus qualified-bidder logic.
- Developed skills in multi-format report generation and audit-trail design for a governance-sensitive application.

12. Operational Safety and Governance Guardrails

1. **Human-in-the-Loop Sign-off:** GeM TenderLens is a decision-support system. Final procurement contract awards must be signed off by human procurement officers.
2. **Strict Evidence Traceability:** Every finding includes an exact document citation, consisting of file name, page number, and clause reference.

3. **Review Flags for Low Confidence:** Extraction confidence scores below 0.70, or unverified vendor claims, trigger mandatory human review alerts.
 4. **Isolated Vector Storage:** Each tender workspace operates within a strictly isolated ChromaDB collection, preventing cross-tender data leakage.
 5. **Immutable Audit Trail:** All key actions, including intake, search, evaluation runs, and report exports, produce timestamped audit logs.
-

13. Conclusion

GeM TenderLens demonstrates how retrieval-augmented generation and multi-agent AI can automate and improve the evaluation of government procurement tenders. By combining a clause-aware document ingestion pipeline, a ChromaDB-based knowledge base, a CrewAI multi-agent evaluation engine with a deterministic rule-based fallback, and a governance-focused audit and export layer, the platform provides a reliable and evidence-backed solution for comparing vendor proposals against tender requirements.

The system enables procurement committees to ingest tender and vendor documents automatically, verify technical compliance with exact citations, normalize commercial pricing, identify risks and missing documentation, and generate committee-ready evaluation reports. This project highlights the potential of retrieval-augmented, multi-agent systems to support faster, more consistent, and more auditable decision-making in high-stakes procurement processes, while keeping the final award decision firmly in the hands of the human committee.